

JMPM

Referência da API - RPO Bytecode Injector

DATA

30/09/2026

ARQUIVO

API-REFERENCE.md

Package: github.com/advpl/compiler/pkg/rpo

Funções Principais

Parse

Parseia um arquivo RPO bruto e retorna uma estrutura `RPOFile`.

```
func Parse(data []byte) (*RPOFile, error)
```

Parâmetros:

- `data`: Bytes brutos do arquivo RPO

Retorno:

- `*RPOFile`: Estrutura parseada
- `error`: Erro se o RPO for inválido

Exemplo:

```
data, _ := os.ReadFile("custom.rpo")
f, err := rpo.Parse(data)
if err != nil {
    log.Fatal(err)
}
fmt.Printf("Nome: %s, Body: %d bytes\n", f.Name, len(f.Body))
```

SelfOffset

Retorna o self offset do RPO.

```
func SelfOffset(data []byte) uint32
```

Uso:

```
offset := rpo.SelfOffset(data)
```

Type: RPOFile

Estrutura que representa um arquivo RPO parseado.

```
type RPOFile struct {
    Name          string
    SelfOffset    uint32
    AdminSection  []byte
    Body          []byte
    FooterMagic   string
    Trailer       []byte
}
```

Campos:

- `Name`: Nome do RPO (ex: "custom", "tlpp")
- `SelfOffset`: Offset onde o body começa
- `AdminSection`: Dados admin (opacos)
- `Body`: Corpo criptografado
- `FooterMagic`: Magic bytes (ex: "APNSRM0419")
- `Trailer`: 24 bytes de trailer

Package: github.com/advpl/compiler/pkg/rpo/injector

NewInjector

Cria um novo injector a partir de dados brutos.

```
func NewInjector(data []byte) (*Injector, error)
```

Exemplo:

```
inj, err := injector.NewInjector(rpoData)
if err != nil {
    log.Fatal(err)
}
```

LoadCapture

Carrega segmentos de captura de um arquivo JSON.

```
func (inj *Injector) LoadCapture(path string) error
```

Exemplo:

```
err := inj.LoadCapture("capture.json")
```

DecodeBody

Decodifica o body usando os segmentos de captura.

```
func (inj *Injector) DecodeBody() ([]byte, map[int][]byte)
```

Retorno:

- `[]byte`: Body original
- `map[int][]byte`: Map de offset -> plaintext decodificado

GetAPORecords

Extrai registros APO dos bodies decomprimidos.

```
func (inj *Injector) GetAPORecords() []*APORecord
```

ReplaceAPO

Substitui o código de um registro APO.

```
func (inj *Injector) ReplaceAPO(recordIdx int, newCode []byte) error
```

Parâmetros:

- `recordIdx`: Índice do registro (0-based)
- `newCode`: Novo bytecode

Exemplo:

```
newCode := make([]byte, 50)
// ... preencher código ...
err := inj.ReplaceAPO(0, newCode)
```

InjectBytecode

Injeta bytecode de um arquivo em um registro APO.

```
func (inj *Injector) InjectBytecode(recordIdx int, bytecodePath string) error
```

Save

Salva o RPO modificado.

```
func (inj *Injector) Save(path string) error
```

Type: APORecord

```
type APORecord struct {
    Offset    int
    Size      int
    Name      string
    Timestamp float64
    BuildType int
    BinaryType int
    Code      []byte
}
```

Type: CaptureSegment

```
type CaptureSegment struct {
    N          int
    Cipher     string
    Key        string
    IV         string
    Plaintext  string
}
```

Package: github.com/advpl/compiler/pkg/rpo

CipherHeuristic

```
type CipherHeuristic struct {
    // exported fields...
}

func NewCipherHeuristic() *CipherHeuristic
func (h *CipherHeuristic) Identify(keyHex, ivHex string) (*CipherInfo, error)
```

CaptureParser

```
type CaptureParser struct {
    // exported fields...
}

func NewCaptureParser() *CaptureParser
func (p *CaptureParser) LoadFile(path string) error
func (p *CaptureParser) IdentifyCiphers() error
func (p *CaptureParser) GetSegments() []CaptureSegment
```

EncryptSegment / DecryptSegment

```
func EncryptSegment(cipherName string, key, iv, plaintext []byte) ([]byte, error)
func DecryptSegment(cipherName string, key, iv, ciphertext []byte) ([]byte, error)
```

CLI Reference

advplc rpo info

```
./advplc rpo info arquivo.rpo
```

Output:

```
Arquivo .....: arquivo.rpo
Tamanho .....: 21118 bytes
Nome do RPO ....: custom
SelfOffset .....: 374 (0x176)
Admin section ..: 336 bytes
Body .....: 20710 bytes
Footer magic ...: APNSRM0419
```

advplc rpo inject

```
./advplc rpo inject <rpo> <captura> [opções]
```

Opções:

- `--list`: Listar segmentos APO
- `--inject N=arquivo`: Injetar bytecode no registro N
- `--identify`: Usar heurística
- `-o, --output`: Arquivo de saída
- `--verbose`: Modo verbose

Exemplo:

```
./advplc rpo inject custom.rpo capture.json \  
  --inject 1=bytecode.bytecode \  
  -o output.rpo \  
  --verbose
```

advplc rpo decrypt

```
./advplc rpo decrypt <rpo> <captura>
```

Decodifica e exibe conteúdo do RPO.

advplc rpo regions

```
./advplc rpo regions <rpo>
```

Classifica regiões por entropia.

advplc rpo analyze

```
./advplc rpo analyze <rpo>
```

Análise detalhada (entropia, strings, padrões).

Exemplo Completo

```
package main
```

```

import (
    "fmt"
    "os"
    "github.com/advpl/compiler/pkg/rpo"
    "github.com/advpl/compiler/pkg/rpo/injector"
)

func main() {
    // 1. Carregar RPO
    rpoData, _ := os.ReadFile("custom.rpo")

    // 2. Criar injector
    inj, _ := injector.NewInjector(rpoData)
    fmt.Printf("RPO: %s (%d bytes)\n", inj.RPOName(), len(rpoData))

    // 3. Carregar captura
    inj.LoadCapture("capture.json")

    // 4. Decodificar
    body, decoded := inj.DecodeBody()
    fmt.Printf("Segmentos decodificados: %d\n", len(decoded))

    // 5. Extrair APO records
    records := inj.GetAPORecords()
    fmt.Printf("Registros APO: %d\n", len(records))

    // 6. Modificar
    newCode := make([]byte, 50)
    inj.ReplaceAPO(0, newCode)

    // 7. Salvar
    inj.Save("injected.rpo")
    fmt.Printf("Body original: %d bytes\n", len(body))
    fmt.Printf("Body modificado: %d bytes\n", len(inj.Body()))
}

```

Referência da API do RPO Bytecode Injector